UNIDAD ACADEMICA MULTIDISCIPLINARIA MANTE

Materia: Diseño Eléctrico Basado En Sistemas Embebidos

Prácticas de laboratorio: 4 , 5 , 6 , 7 y 8

Equipo:

Badillo Navarro Edgar Santiago,

Sanchez Garcia Jose Arturo

Zapata Echeverria Juan De Dios

Zamittiz Camacho Irving Fernando.

Maestro: López Piña Daniel

Grado: 8 **Grupo:** F

LABORATORIO 04: Sensores Analógicos — ADC, LDR y Potenciómetro

Modalidad: Físico (Protoboard)

Objetivo

Comprender el funcionamiento del conversor analógico-digital (ADC) del microcontrolador, conectar y leer valores de un potenciómetro y una fotorresistencia (LDR), y usar estos valores para controlar un LED y un buzzer.

Desarrollo

Parte 1: Potenciómetro

Se conectó un potenciómetro de 10 kΩ a los pines VCC, GND y la entrada analógica A0. Se cargó un código para leer analogRead(A0), calcular el voltaje y el porcentaje, y mostrarlo en el Monitor Serial y el Plotter Serial. El valor de la lectura controlaba el brillo de un LED en el pin 9 mediante analogWrite().

Codigo De la practica

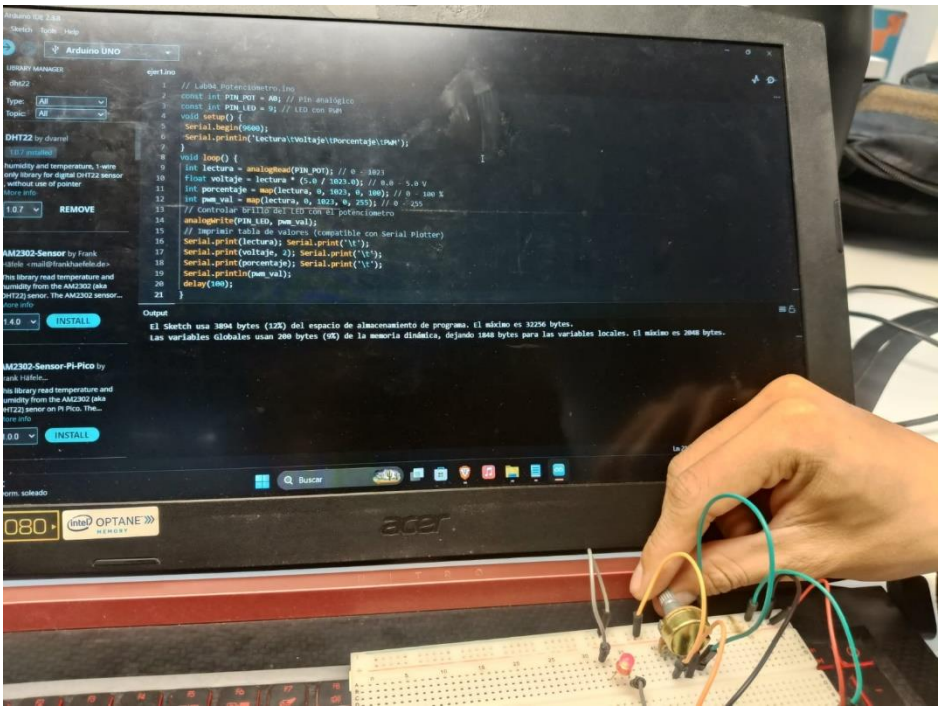
```
// Lab04_Potenciometro.ino

const int PIN_POT = A0; // Pin analógico

const int PIN_LED = 9; // LED con PWM

void setup() {

 Serial.begin(9600);

 Serial.println('Lectura\tVoltaje\tPorcentaje\tPWM');
```

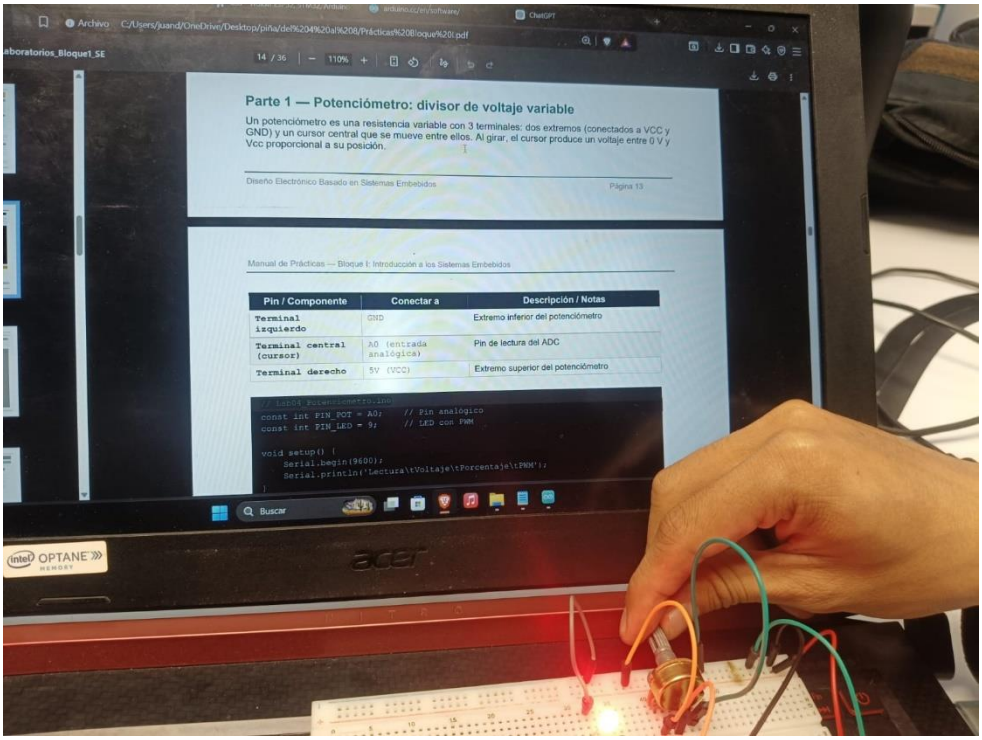
```
}
void loop() {
 int lectura = analogRead(PIN_POT); // 0 - 1023
 float voltaje = lectura * (5.0 / 1023.0); // 0.0 - 5.0 V
 int porcentaje = map(lectura, 0, 1023, 0, 100); // 0 - 100 %
 int pwm_val = map(lectura, 0, 1023, 0, 255); // 0 - 255
 // Controlar brillo del LED con el potenciometro
 analogWrite(PIN_LED, pwm_val);
 // Imprimir tabla de valores (compatible con Serial Plotter)
 Serial.print(lectura); Serial.print('\t');
 Serial.print(voltaje, 2); Serial.print('\t');
 Serial.print(porcentaje); Serial.print('\t');
 Serial.println(pwm_val);
 delay(100);
}
```

Universidad Autónoma de TAMAULIPAS

Unidad Académica Multidisciplinaria Mante

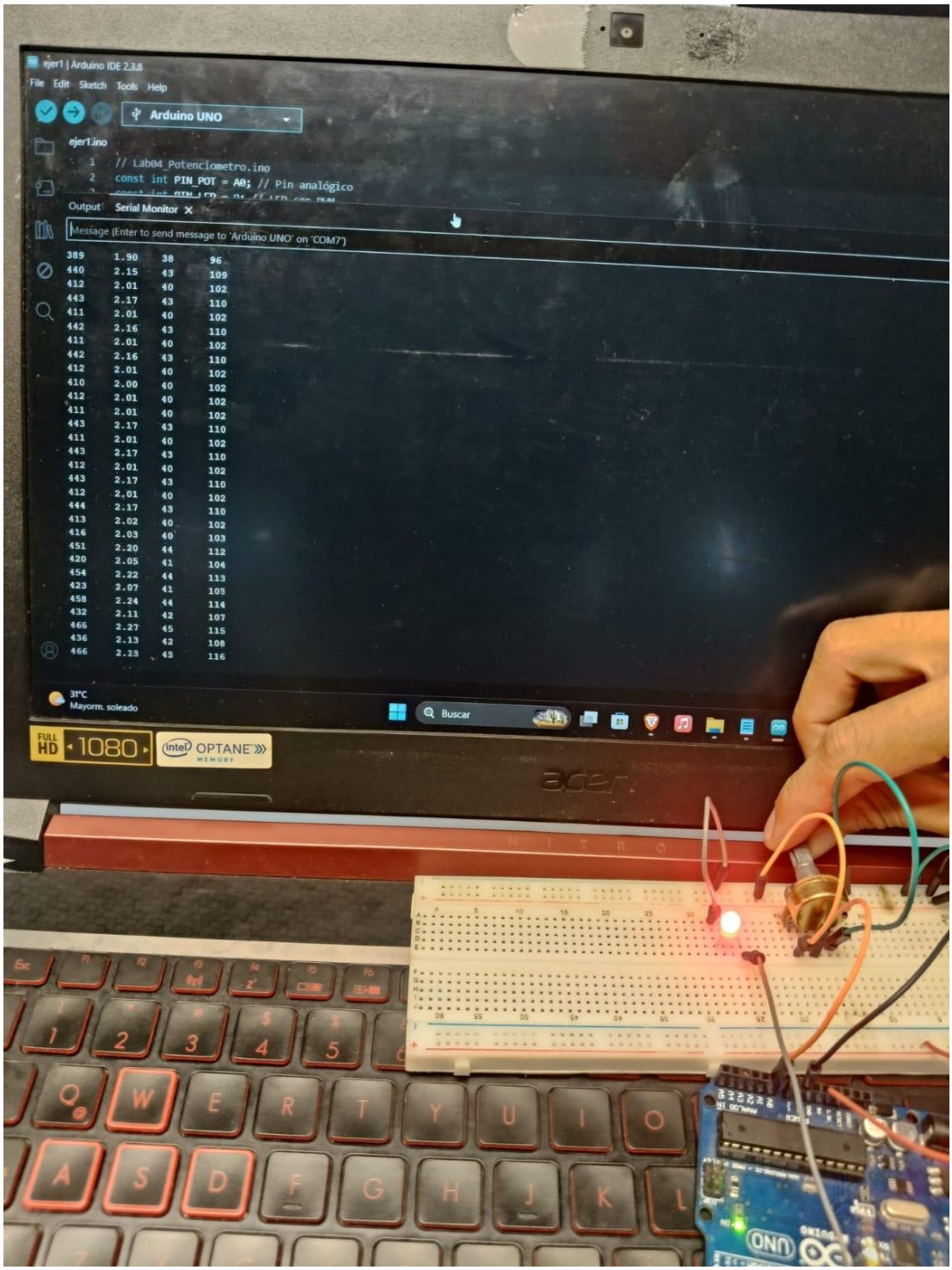
EVIDENCIAS DE PRACTICA LABORATORIO 4





Universidad Autónoma de TAMAULIPAS

Unidad Académica Multidisciplinaria Mante



Parte 2 Ejercicio 2: LDR (Fotorresistencia)

Se construyó un divisor de tensión con la LDR y una resistencia fija de 10 kΩ, conectando el punto medio a la entrada analógica A1. Se programó la lectura de la LDR para controlar el brillo de un LED (a mayor oscuridad, más brillo) y activar una alarma con un buzzer en condiciones de poca luz (<20%).

CODIGO DEL EJERCICIO 2 PRACTICA LABORATORIO 4

```
// Lab04_LDR_Luz.ino

const int PIN_LDR = A1;
const int PIN_LED = 9;
const int PIN_BUZZER = 8;

const int LDR_OSCURO = 800;
const int LDR_LUZ = 100;

void setup() {
  Serial.begin(9600);

  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);
```

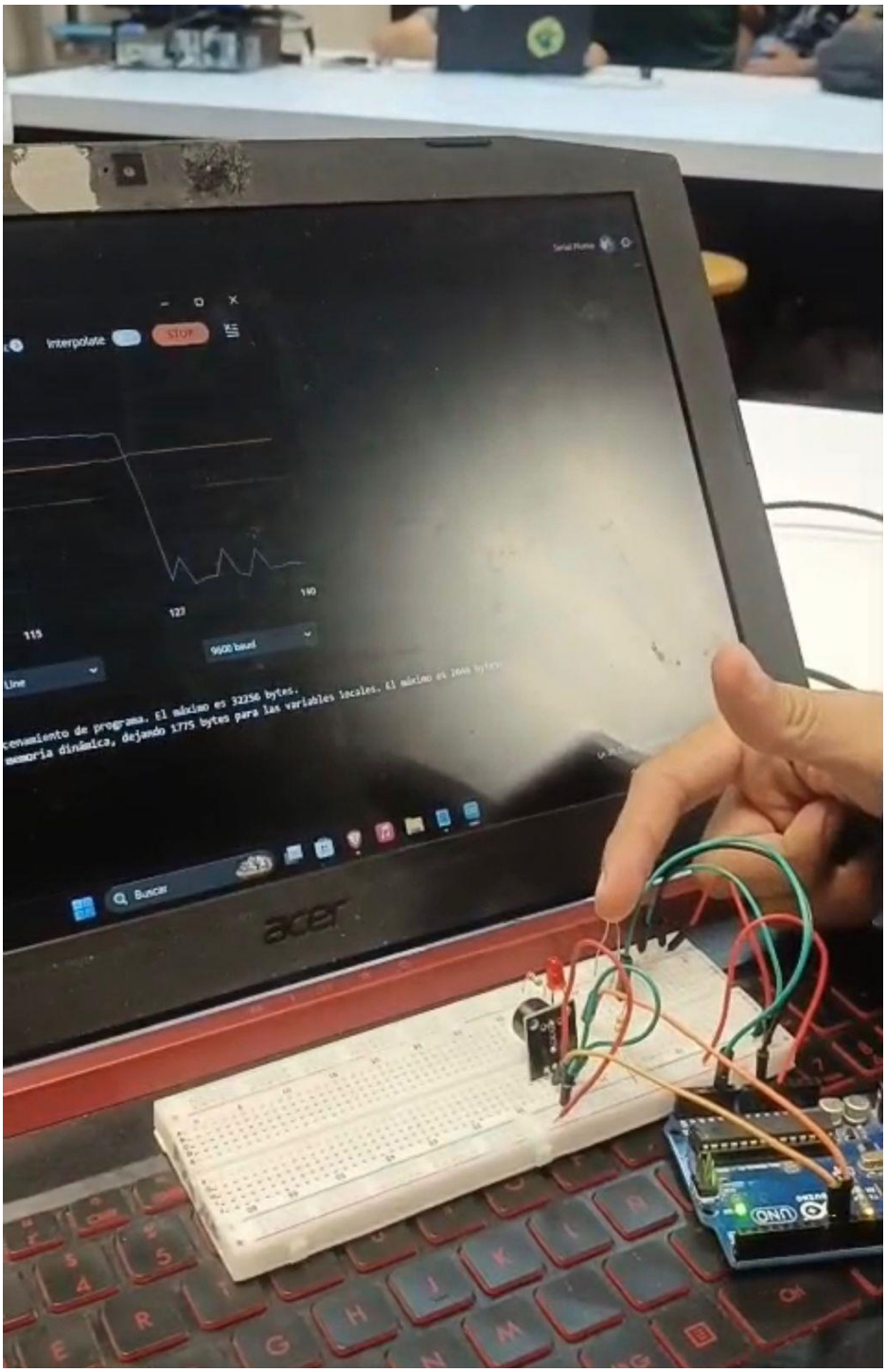
```
  Serial.println("LDR_raw\tLuz%\tEstado");
}

void loop() {
  int lectura_ldr = analogRead(PIN_LDR);

  int luz_pct = map(lectura_ldr, LDR_OSCURO, LDR_LUZ, 0,
100);
  luz_pct = constrain(luz_pct, 0, 100);

  int brillo_led = map(luz_pct, 0, 100, 255, 0);
  analogWrite(PIN_LED, brillo_led);

  const char* estado;

  if (lectura_ldr > 55) {
      estado = "ILUMINADO - BUZZER OFF";
  noTone(PIN_BUZZER);

} else {
```

```
    estado = "MUY OSCURO - BUZZER ON";
  tone(PIN_BUZZER, 1000);
}

  Serial.print(lectura_ldr);
  Serial.print("\t");
  Serial.print(luz_pct);
  Serial.print("\t");
  Serial.println(estado);

  delay(300);
}
```

Universidad Autónoma de TAMAULIPAS

Unidad Académica Multidisciplinaria Mante

EVIDENCIAS DEL EJERCICIO 2 PRACTICA DE LABORATORIO 4



Resultados

- El potenciómetro controló el brillo del LED y el valor se visualizó correctamente en el Monitor Serial y Plotter.
- La LDR respondió a los cambios de luz, controlando el LED de forma inversa.

Observaciones

- La función map() es muy útil para escalar valores del ADC (0-1023) a otros rangos como porcentaje (0-100) o PWM (0-255).
- constrain() es esencial para evitar que los valores mapeados salgan del rango deseado.
- La resistencia fija en el divisor de tensión de la LDR debe elegirse adecuadamente para el rango de luz esperado.

Conclusión

Se comprendió el funcionamiento del ADC de 10 bits del Arduino, permitiendo la lectura de sensores analógicos y su uso para controlar actuadores de forma proporcional, sentando las bases para sistemas de control realimentado.

LABORATORIO 05: Sensor de Temperatura y Humedad DHT22

Modalidad: Simulador (Wokwi)

Objetivo

Instalar y usar una librería externa, conectar y leer datos del sensor digital DHT22, mostrar la temperatura y humedad en el Monitor Serial e implementar un sistema de alerta.

Desarrollo

Se realizó la simulación en Wokwi, agregando un sensor DHT22, un LED rojo, un LED verde y un buzzer. Se instalaron las librerías "DHT sensor library" y "Adafruit Unified Sensor". Se escribió el código para leer la temperatura y humedad cada 2 segundos.

CODIGO DE LA PRACTICA 5

```
#include "DHT.h"


#define PIN_DHT 7

#define TIPO_DHT DHT22


#define LED_VERDE 8

#define LED_ROJO 9

#define BUZZER 6
```

```
DHT dht(PIN_DHT, TIPO_DHT);

// Umbrales
const float TEMP_ALTA = 30.0;
const float TEMP_BAJA = 10.0;
const float HUM_ALTA  = 80.0;

void setup() {
  Serial.begin(9600);

  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_ROJO, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  dht.begin();

  Serial.println("LAB 05");
}

void loop() {
```

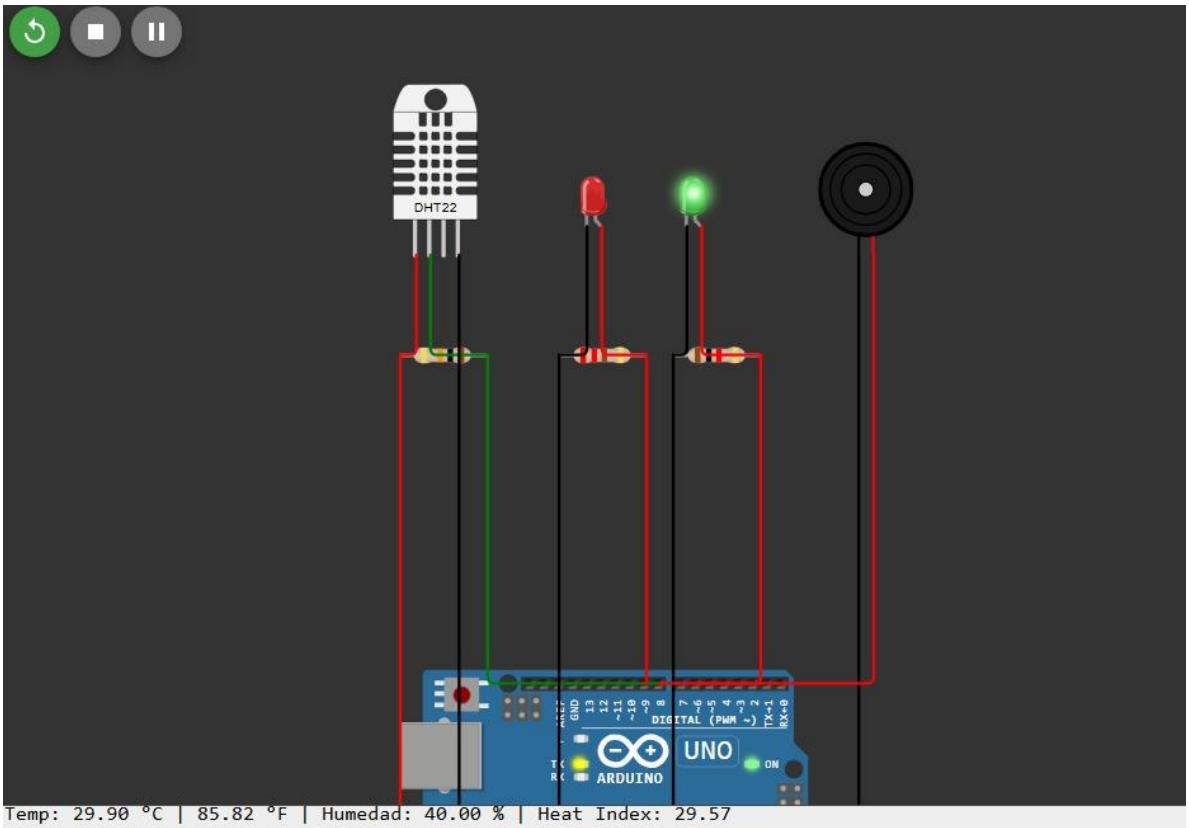
```
  float humedad = dht.readHumidity();
  float tempC = dht.readTemperature();
  float tempF = dht.readTemperature(true);

  // Verificar lectura
  if (isnan(humedad) || isnan(tempC)) {
    Serial.println("Error leyendo DHT22");
    return;
  }

  // Índice de calor
  float indiceCalor = dht.computeHeatIndex(tempC, humedad,
false);

  // Mostrar datos
  Serial.print("Temp: ");
  Serial.print(tempC);
  Serial.print(" °C | ");

  Serial.print(tempF);
  Serial.print(" °F | Humedad: ");
  Serial.print(humedad);
```

```
  Serial.print(" % | Heat Index: ");
  Serial.println(indiceCalor);

  // Lógica de alertas
  if (tempC > TEMP_ALTA) {
    digitalWrite(LED_ROJO, HIGH);
    digitalWrite(LED_VERDE, LOW);
    tone(BUZZER, 1000, 200);

  } else if (tempC < TEMP_BAJA) {
    digitalWrite(LED_ROJO, HIGH);
    digitalWrite(LED_VERDE, LOW);
    tone(BUZZER, 500, 200);

  } else {
    digitalWrite(LED_VERDE, HIGH);
    digitalWrite(LED_ROJO, LOW);
    noTone(BUZZER);
  }
  delay(2000); // IMPORTANTE: DHT22 necesita 2s
}
```

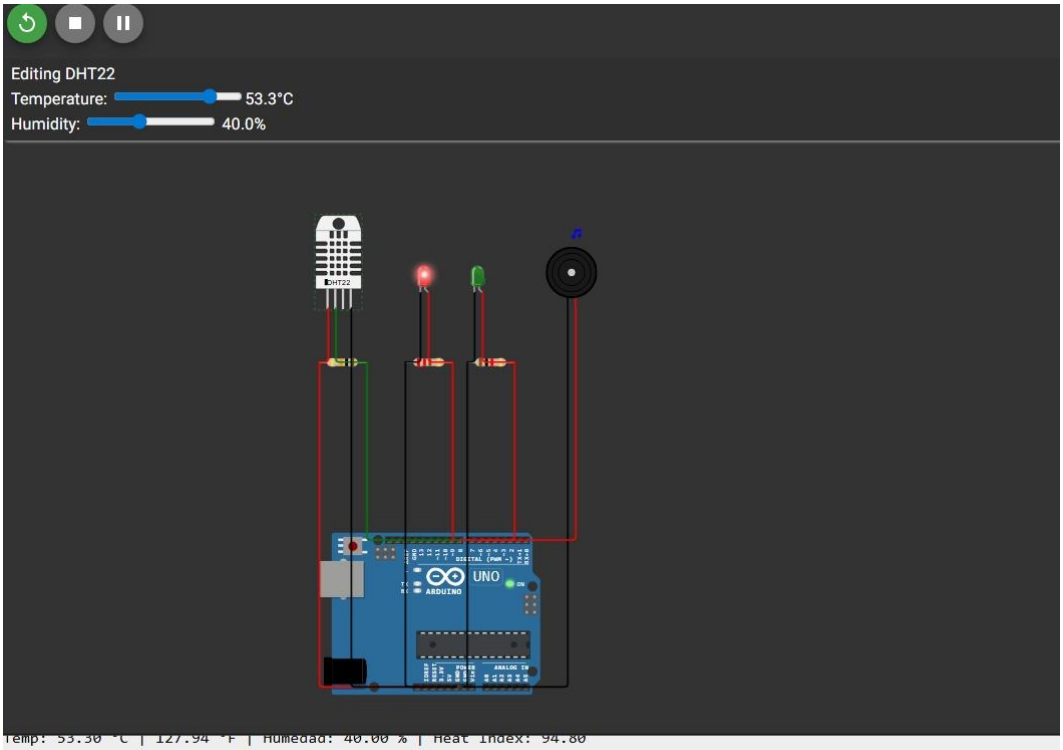
Ejercicio 1: Lectura de Datos Básica

Descripción: Se programó el Arduino para leer la temperatura en °C y °F, y la humedad relativa. Los datos se mostraron en el Monitor Serial con un formato tabulado.

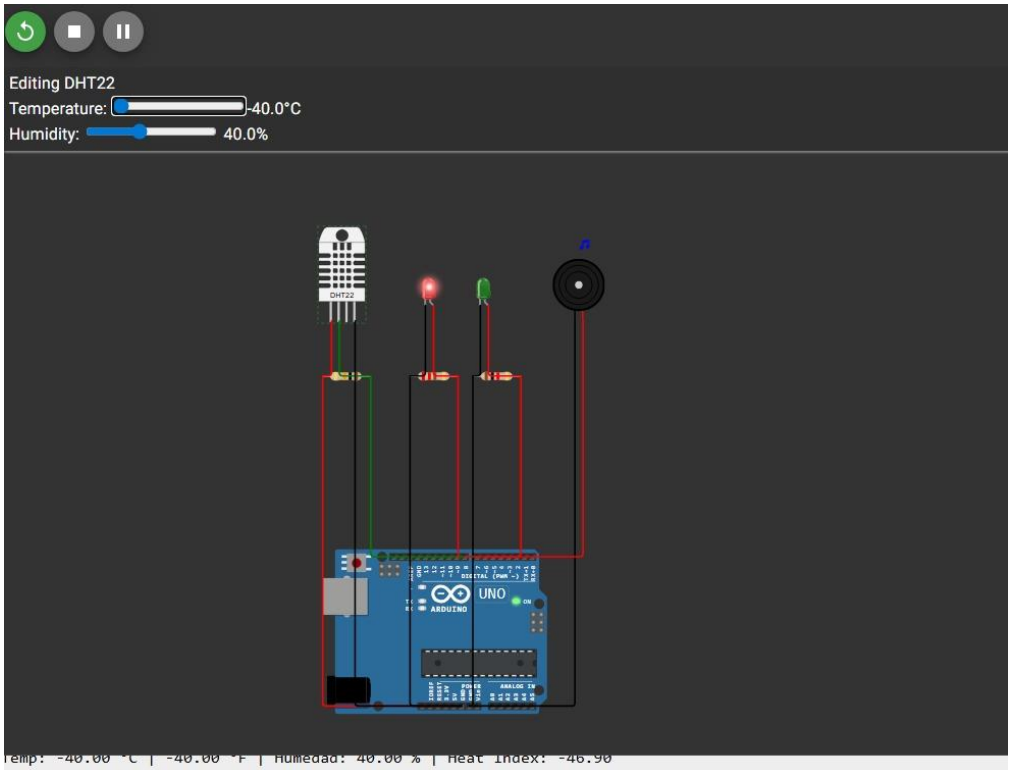
Resultado: El Monitor Serial mostró valores coherentes de temperatura (aprox. 20-30°C) y humedad, actualizándose periódicamente.




```
Temp: 29.90 °C | 85.82 °F | Humedad: 40.00 % | Heat Index: 29.57
Temp: 29.90 °C | 85.82 °F | Humedad: 40.00 % | Heat Index: 29.57
Temp: 29.90 °C | 85.82 °F | Humedad: 40.00 % | Heat Index: 29.57
Temp: 29.90 °C | 85.82 °F | Humedad: 40.00 % | Heat Index: 29.57
Temp: 29.90 °C | 85.82 °F | Humedad: 40.00 % | Heat Index: 29.57
Temp: 29.90 °C | 85.82 °F | Humedad: 40.00 % | Heat Index: 29.57
```




```
Temp: 53.30 °C | 127.94 °F | Humedad: 40.00 % | Heat Index: 94.80
Temp: 53.30 °C | 127.94 °F | Humedad: 40.00 % | Heat Index: 94.80
Temp: 53.30 °C | 127.94 °F | Humedad: 40.00 % | Heat Index: 94.80
```




```
Temp: -40.00 °C | -40.00 °F | Humedad: 40.00 % | Heat Index: -46.90
Temp: -40.00 °C | -40.00 °F | Humedad: 40.00 % | Heat Index: -46.90
Temp: -40.00 °C | -40.00 °F | Humedad: 40.00 % | Heat Index: -46.90
```

Resultados

El sensor mostró valores correctos de temperatura y humedad.

El monitor serial presentó los datos en tiempo real.

El sistema respondió correctamente a los cambios de temperatura y humedad activando alertas.

Conclusión

Se logró comprender el uso de sensores digitales y librerías en Arduino, así como la importancia de interpretar datos ambientales y generar respuestas automáticas.

LABORATORIO 06: Actuadores — Motor DC y Servomotor

Modalidad: Físico (Protoboard)

Objetivo

Entender la necesidad de usar un driver de motor, controlar la velocidad y dirección de un motor DC con PWM y un puente H (L298N), y controlar la posición angular de un servomotor con un potenciómetro.

Desarrollo

Parte 1: Motor DC con Driver L298N

Se armó el circuito con un motor DC pequeño y el módulo L298N, alimentado con una fuente externa. Se programaron las funciones motorAdelante(), motorAtras(), motorFreno() y motorLibre(). Se realizó una prueba de barrido de velocidad.

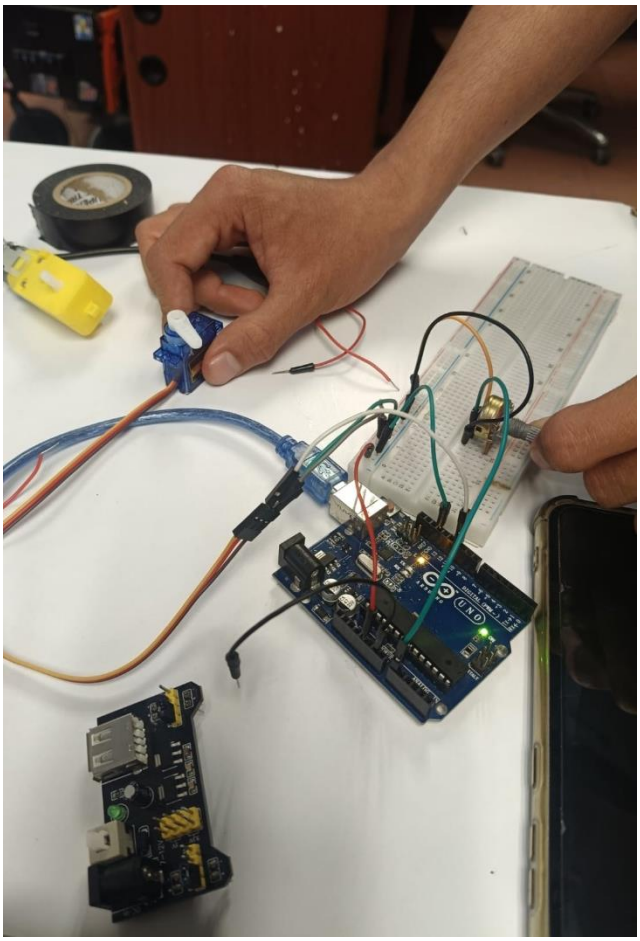
CODIGO EJERICIO 1 PRACTICA LABORATORIO 6

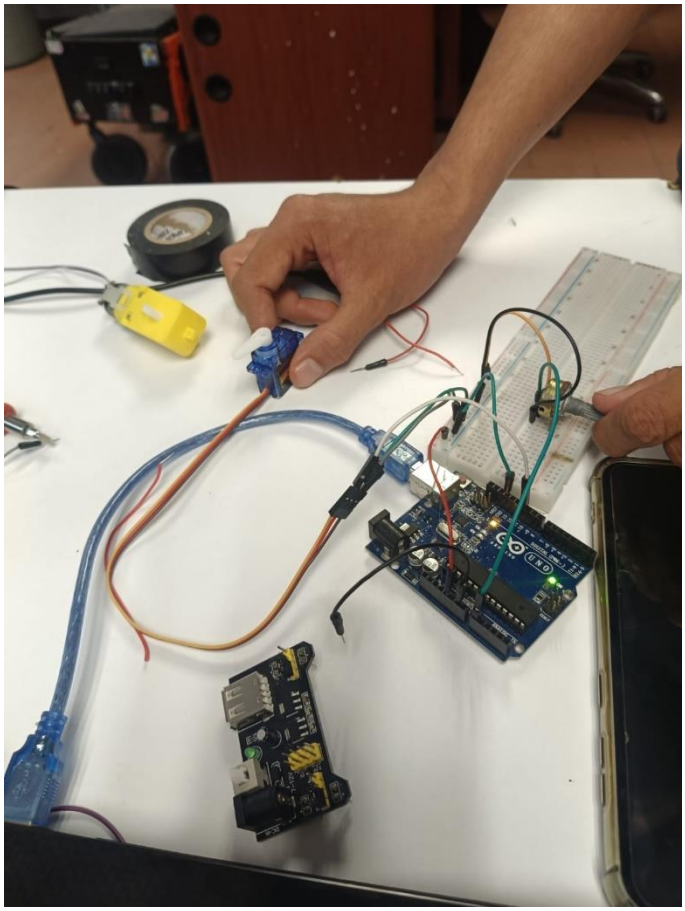
```
// Lab06_Motor_DC.ino
// Control Motor DC con L298N
const int ENA = 10; // PWM velocidad
const int IN1 = 8; // Direccion bit 1
const int IN2 = 9; // Direccion bit 2
void motorAdelante(int velocidad) {
 // velocidad: 0-255
 digitalWrite(IN1, HIGH);
 digitalWrite(IN2, LOW);
 analogWrite(ENA, velocidad);
}
void motorAtras(int velocidad) {
 digitalWrite(IN1, LOW);
 digitalWrite(IN2, HIGH);
 analogWrite(ENA, velocidad);
}
void motorFreno() {
 digitalWrite(IN1, HIGH);
 digitalWrite(IN2, HIGH);
```

```
 analogWrite(ENA, 255); // Freno electromagnetico
}
void motorLibre() {
 analogWrite(ENA, 0); // Motor en rueda libre
}
void setup() {
 pinMode(ENA, OUTPUT);
 pinMode(IN1, OUTPUT);
 pinMode(IN2, OUTPUT);
 Serial.begin(9600);
 Serial.println('Lab 06 — Control Motor DC');
}
void loop() {
 Serial.println('Adelante: velocidad creciente...');
 for (int v = 100; v <= 255; v += 10) {
 motorAdelante(v);
 Serial.print(' Velocidad: '); Serial.println(v);
 delay(300);
 }
 motorLibre(); delay(500);
 Serial.println('Atras: velocidad media...');
```

```
 motorAtras(180); delay(2000);
 motorFreno(); delay(500);
 delay(1000);
}
```

Resultado: El motor giró en ambas direcciones. Se observó que la velocidad aumentaba conforme el valor de PWM era mayor. El freno electromagnético detuvo el motor de forma más brusca que el modo libre.





PRÁCTICA 7

Comunicación I2C con BMP280 y Pantalla OLED (Wokwi)

Objetivo

Implementar comunicación I2C para obtener y mostrar datos de sensores en una pantalla.

Desarrollo / Ejercicios

Se realizó un escaneo del bus I2C para identificar dispositivos.

Se conectó el sensor BMP280 y la pantalla OLED.

Se programó la lectura de:

- Temperatura
- Presión
- Altitud

Se mostraron los datos en la pantalla OLED.

CODIGO PRACTICA 7 Ejercicio 1

```
#include <Wire.h>

#include <Adafruit_BMP085.h>   // ← ESTA ES LA CORRECTA PARA BMP180

#include <Adafruit_SSD1306.h>

#include <Adafruit_GFX.h>
```

```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_ADDR 0x3C

Adafruit_BMP085 bmp;
Adafruit_SSD1306 display(SCREEN_WIDTH,
SCREEN_HEIGHT, &Wire, -1);

void setup() {
  Serial.begin(9600);

  // Inicializar OLED
  if (!display.begin(SSD1306_SWITCHCAPVCC,
OLED_ADDR)) {
    Serial.println("Error OLED");
    while (true);
  }

  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(15, 25);
```

```
  display.println("Iniciando...");
  display.display();

  // Inicializar BMP180
  if (!bmp.begin()) {
    Serial.println("Error BMP180");
    display.clearDisplay();
    display.setCursor(0, 25);
    display.println("ERROR BMP180");
    display.display();
    while (true);
  }

  Serial.println("Sistema listo");
}

void loop() {
  float temp = bmp.readTemperature();      // °C
  float pres = bmp.readPressure() / 100.0; // hPa
  float alt  = bmp.readAltitude();         // metros
```

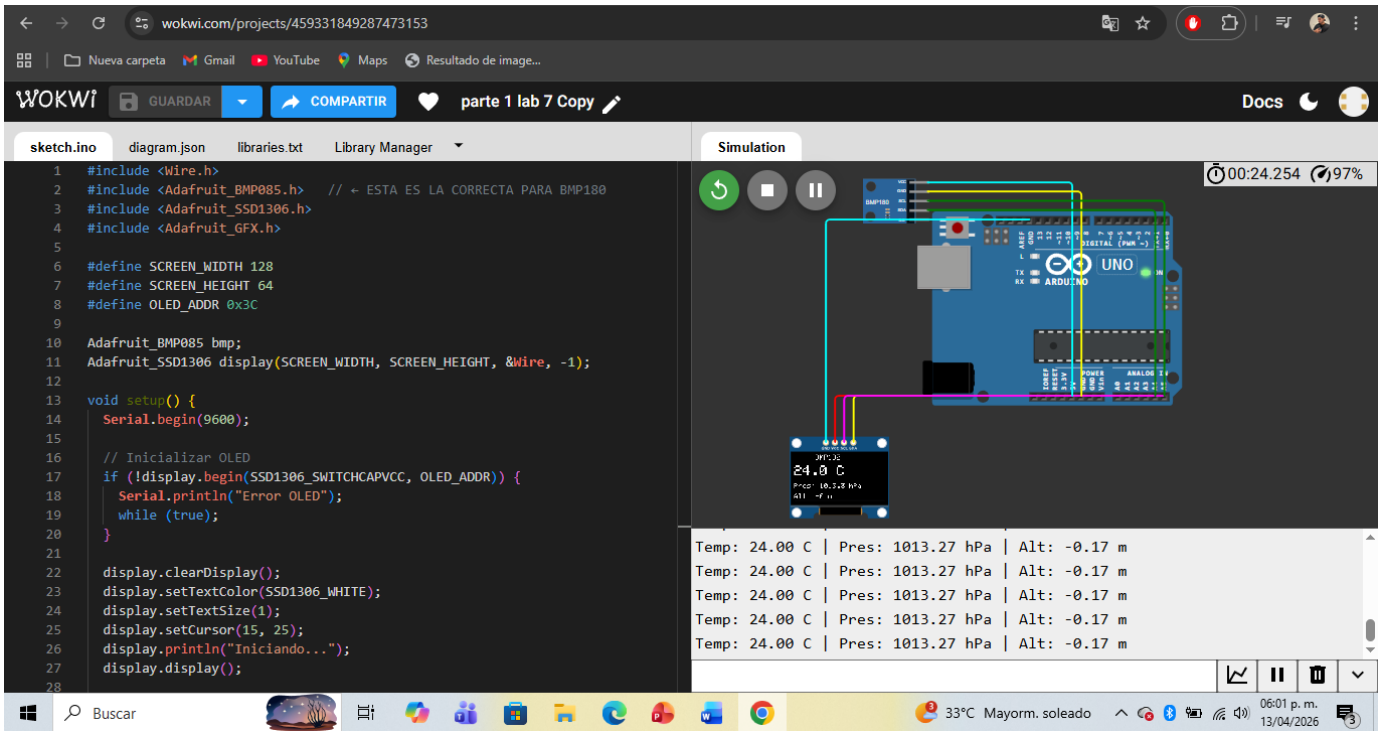
```
display.clearDisplay();

// Título
display.setTextSize(1);
display.setCursor(30, 0);
display.println("BMP180");

// Temperatura grande
display.setTextSize(2);
display.setCursor(0, 15);
display.print(temp, 1);
display.println(" C");

// Presión
display.setTextSize(1);
display.setCursor(0, 40);
display.print("Pres: ");
display.print(pres, 1);
display.println(" hPa");

// Altitud
```

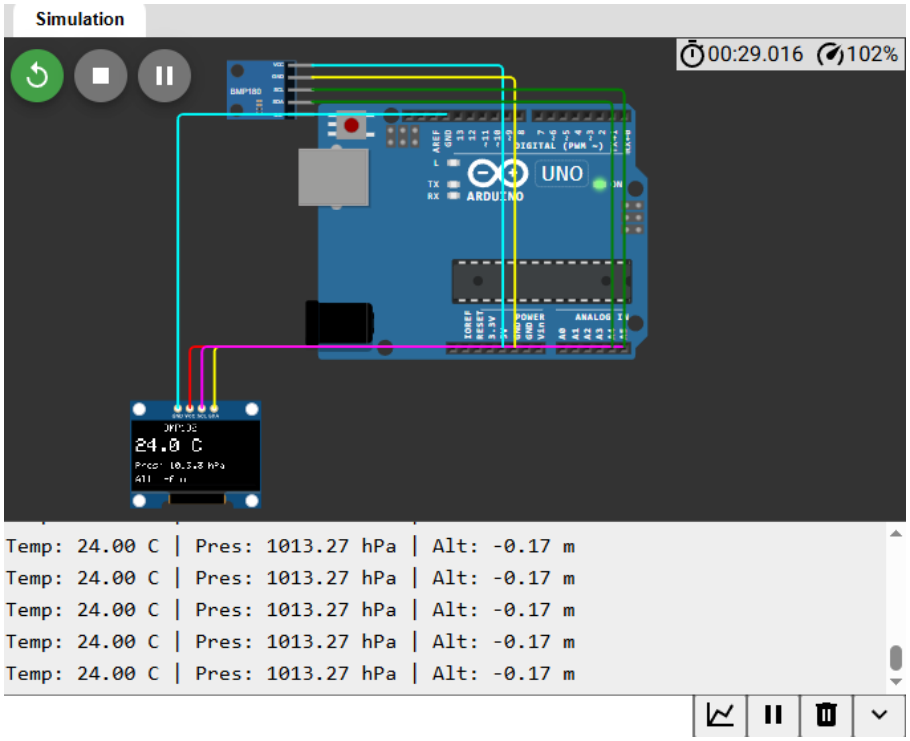
```
  display.setCursor(0, 55);
  display.print("Alt: ");
  display.print(alt, 0);
  display.println(" m");

  display.display();

  // Serial
  Serial.print("Temp: ");
  Serial.print(temp);
  Serial.print(" C | Pres: ");
  Serial.print(pres);
  Serial.print(" hPa | Alt: ");
  Serial.print(alt);
  Serial.println(" m");

  delay(1000);
}
```

Universidad Autónoma de TAMAULIPAS

Unidad Académica Multidisciplinaria Mante

RESULTADOS EJERICIO 1 PRACTICA 7





Resultados ejercicio 1 practica 7

El sistema detectó correctamente los dispositivos I2C.

La pantalla OLED mostró los datos en tiempo real.

Los valores obtenidos fueron coherentes con el entorno.

Conclusión

Se aprendió el funcionamiento del protocolo I2C y cómo conectar múltiples dispositivos en un mismo bus, facilitando la comunicación entre componentes.

Desarrollo Ejercicio 2 Estación de Monitoreo Practica 7

Se utilizó Wokwi para conectar el sensor BMP280 y la pantalla OLED al Arduino mediante comunicación I2C. Se configuraron las librerías necesarias y se inicializaron ambos dispositivos. Posteriormente, se programó la lectura de temperatura, presión y altitud, mostrando los datos en la pantalla OLED y en el monitor serial en tiempo real.

CODIGO EJERCICIO 2 PRACTICA 7

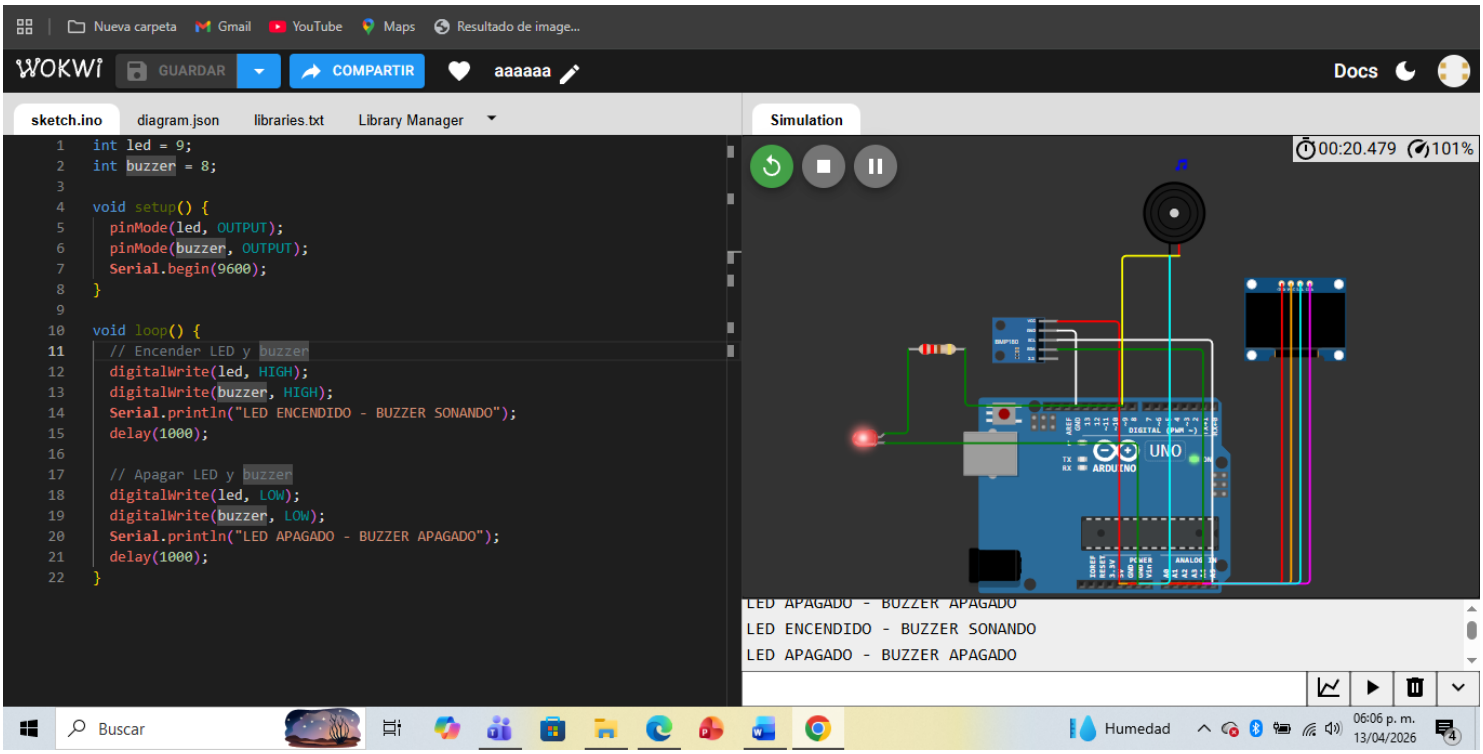
```
int led = 9;

int buzzer = 8;


void setup() {

  pinMode(led, OUTPUT);
```

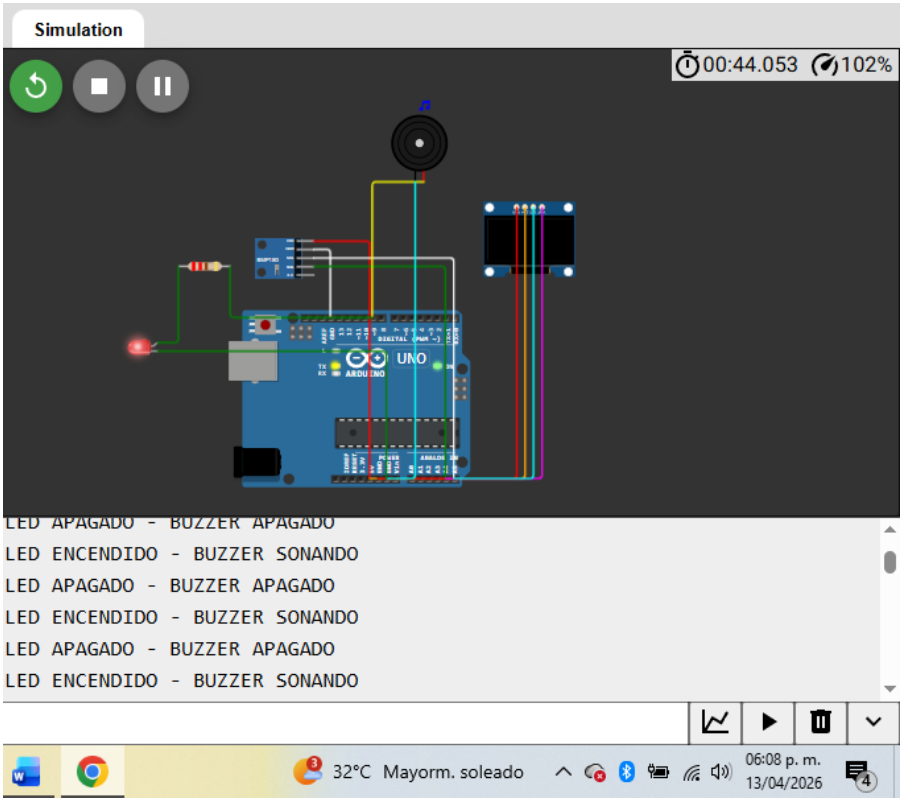
```
  pinMode(buzzer, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  // Encender LED y buzzer
  digitalWrite(led, HIGH);
  digitalWrite(buzzer, HIGH);
  Serial.println("LED ENCENDIDO - BUZZER SONANDO");
  delay(1000);

  // Apagar LED y buzzer
  digitalWrite(led, LOW);
  digitalWrite(buzzer, LOW);
  Serial.println("LED APAGADO - BUZZER APAGADO");
  delay(1000);
}
```

Universidad Autónoma de TAMAULIPAS

Unidad Académica Multidisciplinaria Mante

RESULTADOS EJECICIO 2 PRACTICA 7





PRÁCTICA 8 Comunicación SPI y Proyecto Integrador (Físico en laboratorio)

Objetivo

Comprender el funcionamiento del protocolo SPI y desarrollar un sistema embebido completo que permita leer datos de sensores, mostrarlos en una pantalla y almacenarlos en una memoria SD.

Desarrollo

Se realizó la conexión física del módulo lector de tarjeta SD al Arduino utilizando el protocolo SPI (pines MOSI, MISO, SCK y CS). Posteriormente, se programó la inicialización de la tarjeta SD y se verificó su funcionamiento listando los archivos existentes. Finalmente, se creó un archivo de prueba donde se escribieron datos para comprobar el correcto almacenamiento.

CODIGO DEL EJERCIO 1 PRACTICA 8

```
#include <SPI.h>

#include <SD.h>


const int PIN_CS_SD = 10;


void setup() {

  Serial.begin(9600);
```

```
  // Cambiamos ' ' por " "
  Serial.print("Inicializando SD...");

  if (!SD.begin(PIN_CS_SD)) {
    Serial.println(" ERROR. Verifica conexiones y formato
FAT32.");
    return;
  }
  Serial.println(" OK");

  // Listar archivos
  File root = SD.open("/");
  Serial.println("Archivos en la SD:");

  while (true) {
    File entry = root.openNextFile();
    if (!entry) break;
    Serial.print(" ");
    Serial.print(entry.name());
    Serial.print(" ");
    Serial.println(entry.size());
    entry.close();
```

```
  }

  // Crear archivo
  File archivo = SD.open(" .TXT", FILE_WRITE);
  if (archivo) {
    archivo.println("Lab 08 - Prueba de escritura SD");
    archivo.println("Sistema embebido OK");
    archivo.println("EQUIPO: ");
    archivo.println("BADILLO NAVARRO EDGAR SANTIAGO
");
    archivo.println("SANCHEZ GARCIA JOSE ARTURO ");
    archivo.println("ZAMITTIZ CAMACHO IRVING
FERNANDO");
    archivo.println("ZAPATA ECHVERRIA JUAN DE DIOS");
    archivo.close();
    Serial.println("Archivo EQUIPO.TXT creado
correctamente.");
  }
}

void loop() {}
```

Universidad Autónoma de TAMAULIPAS

Unidad Académica Multidisciplinaria Mante

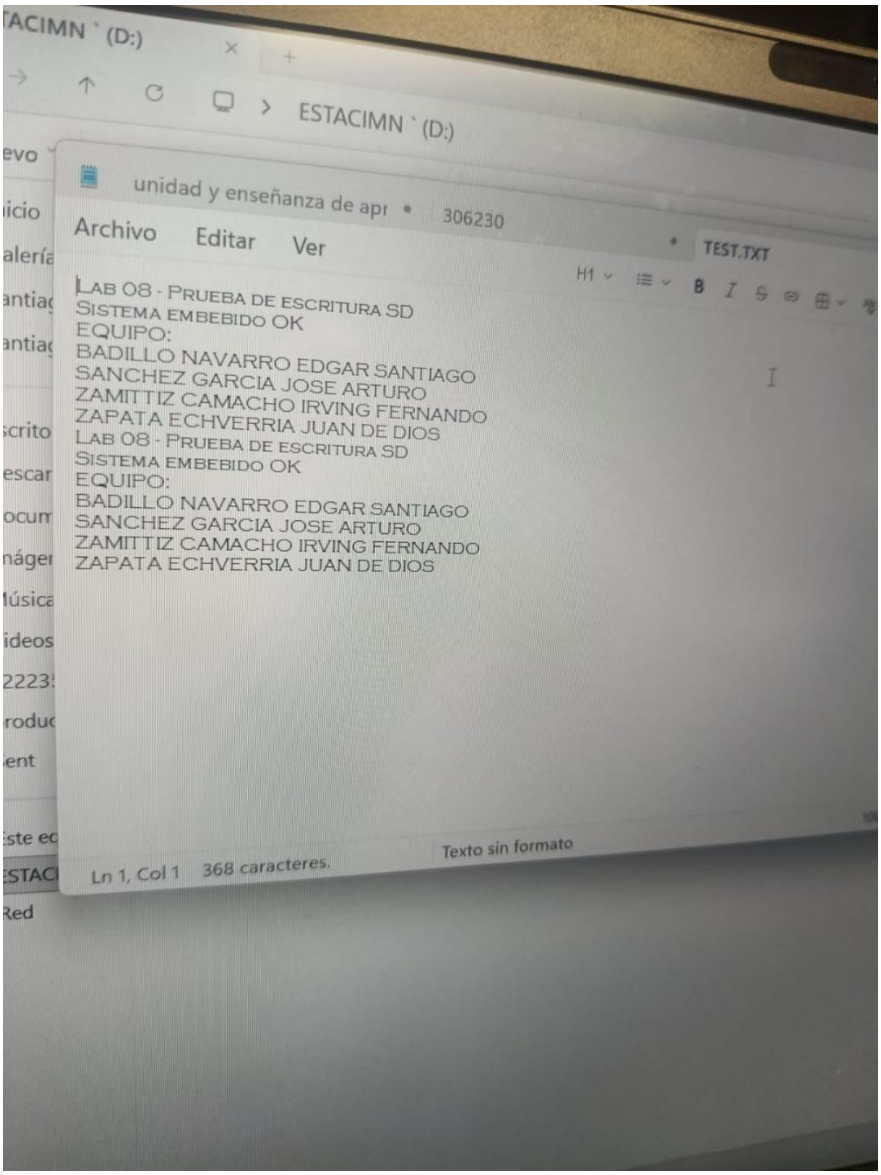
RESULTADOS EJERCICIO 1 PRACTICA 8

```
    Serial.print(" ");
    Serial.println(entry.size());
    entry.close();
  }

  // Crear archivo
  File archivo = SD.open("TEST.TXT", FILE_WRITE);
  if (archivo) {
    archivo.println("Lab 08 - Prueba de escritura SD");
    archivo.println("Sistema embebido OK");
    archivo.println("EQUIPO: ");
    archivo.println("BADILLO NAVARRO EDGAR SANTIAGO ");
    archivo.println("SANCHEZ GARCIA JOSE ARTURO ");
    archivo.println("ZAMITTIZ CAMACHO IRVING FERNANDO");
    archivo.println("ZAPATA ECHVERRIA JUAN DE DIOS");
    archivo.close();
    Serial.println("Archivo EQUIPO.TXT creado correctamente.");
  }
}

void loop() {}
```

Output Serial Monitor ✕

Message (Enter to send message to 'Arduino UNO' on 'COM7')

```
Inicializando SD... OK
Archivos en la SD:
Archivo TEST.TXT creado correctamente.
```



Resultados

La tarjeta SD se inicializó correctamente.

Se detectaron los archivos dentro de la memoria.

Se creó y guardó un archivo de texto sin errores.

Conclusión

Se logró comprender el uso del protocolo SPI para la comunicación con dispositivos de almacenamiento, verificando que es posible guardar información desde Arduino hacia una memoria externa.

EJERICIO 2 PRACTICA 8 Estación de Monitoreo

Se integró un sistema completo en físico utilizando Arduino, sensor DHT22, pantalla OLED, módulo SD, LEDs y un botón. Se programó una máquina de estados con tres modos: MONITOR, ALERTA y LOG. El sistema lee temperatura y humedad, muestra los datos en la pantalla OLED y guarda la información en la tarjeta SD cada cierto tiempo. Además, el sistema cambia de estado según las condiciones ambientales o la interacción con el botón.

CODIGO EJERICIO 2 PRACTICA 8

```
#include <Wire.h>

#include <Adafruit_GFX.h>

#include <Adafruit_SSD1306.h>

#include <DHT.h>

#include <Adafruit_BMP280.h>

#include <SD.h>

#include <SPI.h>
```

```
// ===================== PINES
=====================

#define SCREEN_WIDTH 128

#define SCREEN_HEIGHT 64

#define OLED_ADDR 0x3C

#define SDA_PIN 21

#define SCL_PIN 22


#define DHTPIN 4

#define DHTTYPE DHT22


#define SD_CS_PIN 5       // Chip Select de la SD


#define BUZZER_PIN 25

#define BTN_PIN 34        // Botón (INPUT con pull-up externo
recomendado)


#define LED_VERDE 26

#define LED_ROJO  27

#define LED_AZUL  14
```

```
// ===================== OBJETOS =====================
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
DHT dht(DHTPIN, DHTTYPE);
Adafruit_BMP280 bmp;

// ===================== FSM =====================
enum Estado { ESTADO_MONITOR, ESTADO_ALERTA, ESTADO_LOG };
Estado estadoActual = ESTADO_MONITOR;

// ===================== TIEMPOS =====================
unsigned long tiempoAnterior    = 0;
const long intervaloGuardado    = 5000;   // guardar CSV cada 5 s
const long intervaloLectura     = 2000;   // leer sensores cada 2 s

unsigned long tiempoBoton       = 0;
```

```
const long tiempoLongPress      = 3000;   // 3 s pulsado → ESTADO_LOG

unsigned long tiempoParpadeo    = 0;
const long intervaloParpadeo    = 400;
bool ledRojoEstado              = false;

unsigned long tiempoUltimaLectura = 0;

// ===================== DATOS =====================
float humedad   = 0;
float tempDHT   = 0;
float presion   = 0;
long  totalRegistros = 0;

bool btnPresionado  = false;
bool enLongPress    = false;

const char* nombreArchivo = "/datos.csv";
```

```
// ===================== PROTOTIPOS =====================
void leerSensores();
void guardarCSV();
void contarRegistros();
void mostrarMonitor();
void mostrarAlerta();
void mostrarLog();
void manejarLEDs();
void manejarBuzzer();
void manejarBoton();
void apagarLEDs();

// ==================================================================
void setup() {
Serial.begin(115200);
Wire.begin(SDA_PIN, SCL_PIN);

// Pines
pinMode(BTN_PIN,    INPUT_PULLUP);
```

```
pinMode(BUZZER_PIN, OUTPUT);
pinMode(LED_VERDE,  OUTPUT);
pinMode(LED_ROJO,   OUTPUT);
pinMode(LED_AZUL,   OUTPUT);

// OLED
if (!display.begin(SSD1306_SWITCHCAPVCC,
OLED_ADDR)) {
Serial.println("Error OLED");
while (true);
}
display.clearDisplay();
display.setTextColor(WHITE);

// DHT22
dht.begin();

// BMP280
if (!bmp.begin(0x76)) {
Serial.println("No se encontró BMP280");
while (true);
}
```

```
// SD
if (!SD.begin(SD_CS_PIN)) {
Serial.println("Error SD");
// No bloqueamos; el sistema sigue sin SD
} else {
// Escribir cabecera si el archivo es nuevo
if (!SD.exists(nombreArchivo)) {
File f = SD.open(nombreArchivo, FILE_WRITE);
if (f) {

f.println("timestamp_ms,temperatura_C,humedad_%,presion_
hPa");
f.close();
}
}
contarRegistros();
}

Serial.println("Sistema iniciado");
}
```

```
//
==========================================================
void loop() {
unsigned long ahora = millis();

// --- Lectura de sensores periódica ---
if (ahora - tiempoUltimaLectura >= intervaloLectura) {
tiempoUltimaLectura = ahora;
leerSensores();
}

// --- Guardar CSV cada 5 s ---
if (ahora - tiempoAnterior >= intervaloGuardado) {
tiempoAnterior = ahora;
guardarCSV();
}

// --- Manejar botón ---
manejarBoton();

// --- Determinar estado (solo si no estamos en LOG) ---
```

```
if (estadoActual != ESTADO_LOG) {
if (tempDHT > 35.0 || humedad > 85.0) {
estadoActual = ESTADO_ALERTA;
} else {
estadoActual = ESTADO_MONITOR;
}
}

// --- OLED según estado ---
switch (estadoActual) {
case ESTADO_MONITOR: mostrarMonitor(); break;
case ESTADO_ALERTA:  mostrarAlerta();  break;
case ESTADO_LOG:     mostrarLog();     break;
}

// --- LEDs y buzzer ---
manejarLEDs();
manejarBuzzer();

delay(50); // pequeño delay para estabilidad
}
```

```
//
==========================================================
void leerSensores() {
float h = dht.readHumidity();
float t = dht.readTemperature();
float p = bmp.readPressure() / 100.0;

if (!isnan(h)) humedad = h;
if (!isnan(t)) tempDHT = t;
presion = p;

Serial.printf("[Sensor] T:%.1f°C  H:%.1f%%  P:%.1fhPa\n",
tempDHT, humedad, presion);
}

//
==========================================================
void guardarCSV() {
if (isnan(humedad) || isnan(tempDHT)) return;
```

```
File f = SD.open(nombreArchivo, FILE_WRITE | O_APPEND);
if (!f) {
Serial.println("No se pudo abrir CSV");
return;
}
f.printf("%lu,%.2f,%.2f,%.2f\n", millis(), tempDHT, humedad,
presion);
f.close();
totalRegistros++;
Serial.printf("[SD] Registro #%ld guardado\n",
totalRegistros);
}

//
==========================================================
=========
void contarRegistros() {
File f = SD.open(nombreArchivo, FILE_READ);
if (!f) return;
totalRegistros = -1; // descontar cabecera
while (f.available()) {
if (f.read() == '\n') totalRegistros++;
```

```
}
f.close();
if (totalRegistros < 0) totalRegistros = 0;
}

//
==========================================================
=========
void mostrarMonitor() {
display.clearDisplay();
display.setTextSize(1);
display.setCursor(0, 0);
display.println("== MONITOR ==");

display.setCursor(0, 14);
display.print("T: "); display.print(tempDHT, 1); display.println(" C");

display.setCursor(0, 26);
display.print("H: "); display.print(humedad, 1); display.println(" %");
```

```
display.setCursor(0, 38);
display.print("P: "); display.print(presion, 1); display.println(" hPa");

display.setCursor(0, 52);
display.print("Reg: "); display.print(totalRegistros);
display.display();
}

// ==========================================================
void mostrarAlerta() {
display.clearDisplay();
display.setTextSize(1);
display.setCursor(0, 0);
display.println("!! ALERTA !!");

if (tempDHT > 35.0) {
display.setCursor(0, 14);
display.print("TEMP ALTA: ");
display.print(tempDHT, 1);
```

```
display.println(" C");
}
if (humedad > 85.0) {
display.setCursor(0, 26);
display.print("HUM ALTA: ");
display.print(humedad, 1);
display.println(" %");
}
display.setCursor(0, 42);
display.print("P: "); display.print(presion, 1); display.println(" hPa");
display.display();
}

//
==========================================================
void mostrarLog() {
display.clearDisplay();
display.setTextSize(1);
display.setCursor(0, 0);
display.println("== LOG SD ==");
```

```
display.setCursor(0, 20);
display.print("Registros:");

display.setTextSize(2);
display.setCursor(0, 36);
display.print(totalRegistros);
display.display();
}

//
==========================================================
=========
void manejarLEDs() {
unsigned long ahora = millis();
apagarLEDs();

switch (estadoActual) {
case ESTADO_MONITOR:
digitalWrite(LED_VERDE, HIGH);
break;
```

```
case ESTADO_ALERTA:
// Parpadeo rojo
if (ahora - tiempoParpadeo >= intervaloParpadeo) {
tiempoParpadeo = ahora;
ledRojoEstado = !ledRojoEstado;
}
digitalWrite(LED_ROJO, ledRojoEstado ? HIGH : LOW);
break;

case ESTADO_LOG:
digitalWrite(LED_AZUL, HIGH);
break;
}
}

//
==========================================================
=========
void manejarBuzzer() {
if (estadoActual == ESTADO_ALERTA) {
// Tono intermitente
unsigned long ahora = millis();
```

```
static unsigned long tBuzzer = 0;
static bool buzzerOn = false;

if (ahora - tBuzzer >= 500) {
tBuzzer = ahora;
buzzerOn = !buzzerOn;
if (buzzerOn) tone(BUZZER_PIN, 1000);
else noTone(BUZZER_PIN);
}
} else {
noTone(BUZZER_PIN);
}
}

//
==========================================================
=========
void manejarBoton() {
bool btnActual = (digitalRead(BTN_PIN) == LOW); // activo
en LOW

if (btnActual && !btnPresionado) {
```

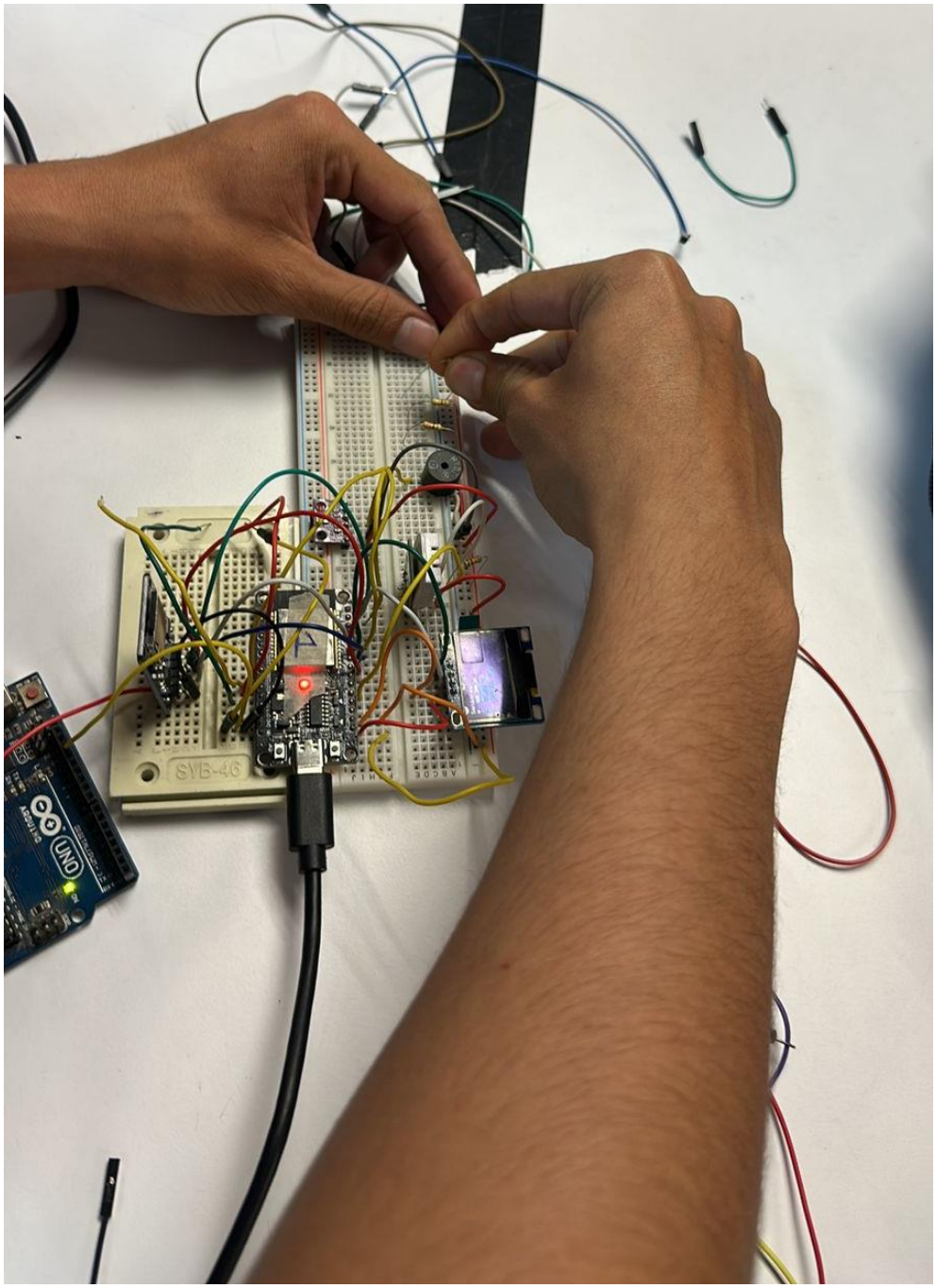
```
// Flanco de bajada: empieza a contar
btnPresionado = true;
enLongPress   = false;
tiempoBoton   = millis();
}

if (btnActual && btnPresionado) {
// Mantenido: verificar long press
if (!enLongPress && (millis() - tiempoBoton >=
tiempoLongPress)) {
enLongPress   = true;
estadoActual  = ESTADO_LOG;
}
}

if (!btnActual && btnPresionado) {
// Flanco de subida: soltar botón
btnPresionado = false;
if (enLongPress) {
// Volver a MONITOR (o ALERTA si condiciones se
mantienen)
estadoActual = ESTADO_MONITOR;
```

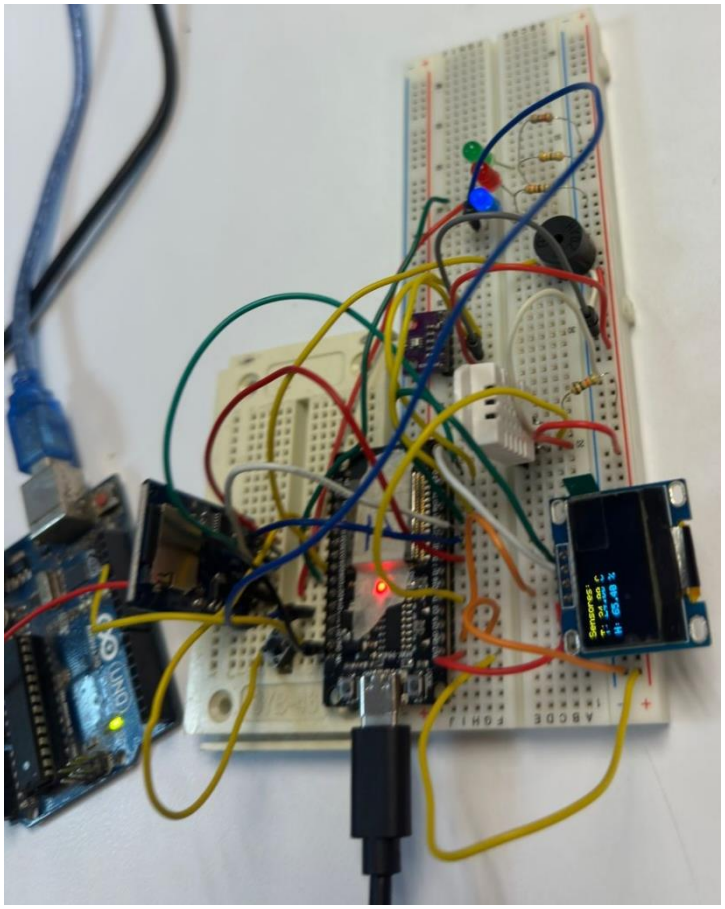
```
enLongPress  = false;
}
}
}

//
==========================================================
=========
void apagarLEDs() {
digitalWrite(LED_VERDE, LOW);
digitalWrite(LED_ROJO,  LOW);
digitalWrite(LED_AZUL,  LOW);
}
```

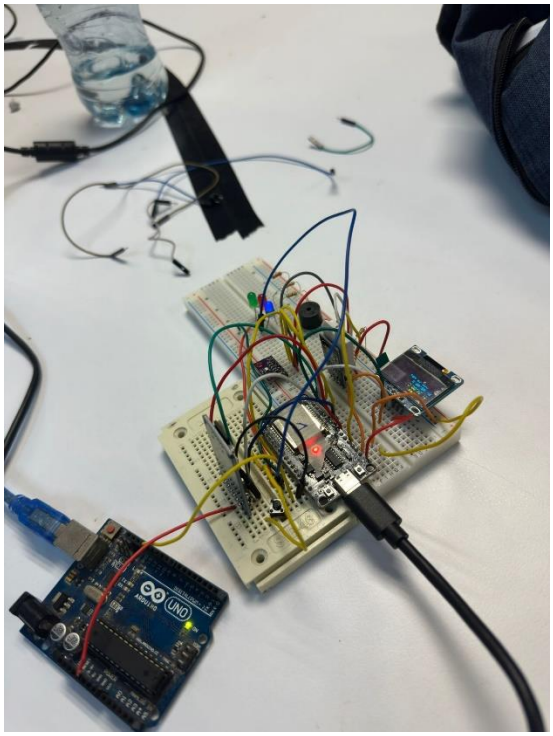
Universidad Autónoma de TAMAULIPAS

Unidad Académica Multidisciplinaria Mante

RESULTADOS DEL EJERCIO 2 PRACTICA 8







Resultados

- El sistema mostró correctamente la temperatura y humedad en la pantalla OLED.
- Los datos se guardaron automáticamente en la tarjeta SD en formato CSV.
- El sistema cambió entre estados (MONITOR, ALERTA, LOG) correctamente.
- Los LEDs y el buzzer respondieron según las condiciones del sistema.

Conclusión de esta practica

Se logró integrar múltiples componentes en un solo sistema funcional, aplicando conocimientos de sensores, comunicación y programación. Esta práctica permitió comprender cómo funcionan los sistemas embebidos en aplicaciones reales como estaciones de monitoreo.

Conclusión Final De Todas Las Practicas

En equipo, durante el desarrollo de las prácticas 4 a la 8, logramos comprender de manera más completa el funcionamiento de los sistemas embebidos, trabajando tanto en simulación como en el laboratorio. A lo largo de estas prácticas utilizamos distintos sensores, actuadores y módulos de comunicación, lo que nos permitió aplicar los conocimientos teóricos en situaciones reales.

También aprendimos a utilizar protocolos como I2C y SPI, así como diferentes librerías de Arduino que facilitan la programación y el control de los dispositivos. El trabajo en equipo fue fundamental, ya que pudimos apoyarnos para resolver errores, mejorar las conexiones y entender mejor cada parte de las prácticas.

En general, estas actividades nos ayudaron a desarrollar habilidades tanto técnicas como colaborativas, permitiéndonos comprender cómo se implementan sistemas de monitoreo y control en la vida real, y preparándonos para proyectos más complejos en el área de sistemas embebidos.